



Fundamentals of Programming: Functions in C++

Bilal Ahmed

Agenda

- Definition of Function
- Declaration of Function
- Calling a Function
- Types of functions
- Function prototype
- Void vs return type
- Formal and Actual Parameters
- Call by value vs call by Reference
- Type casting in functions

Function

3



A **block of code** that performs some specific task.



Problem: Write code to Draw a circle on screen, and color it.



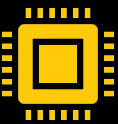
Let's say we need the code for above mentioned task for multiple times at different time instances and with different color

Function

4



Put commonly or repeatedly done tasks together, to avoid writing the same code again and again for different inputs.



A function is a block of code that runs only when it is called, exception is the main function (which is called automatically by OS)

Function

- ▶ A piece of code which takes input, processes it and produces output

5

```
int max(int x, int y)
{
    if (x > y)
        return x;
    else
        return y;
}

// main function that doesn't receive any parameter and
// returns integer
int main()
{
    int a = 10, b = 20;

    // Calling above function to find max of 'a' and 'b'
    int m = max(a, b);

    cout << "m is " << m;
    return 0;
}
```

Advantages of functions

6

Code Organization

Code Readability

Code Reusability

Types of functions

- ▶ Two Types of functions:
 - ▶ Pre-defined/Standard Library Functions
 - ▶ User defined functions

C++ User Defined function

- ▶ C++ allows users to create their own functions
- ▶ User defines a function and groups the code in a block
- ▶ User calls the function and it is executed

C++ function declaration

9

```
returnType functionName (parameter1, parameter2,...) {  
    // function body  
}
```

1. Return Type, Function name are mandatory
2. Parameters are optional

```
// function declaration
void greet() {
    cout << "Hello World";
}
```

Here,

- the name of the function is `greet()`
- the return type of the function is `void`
- the empty parentheses mean it doesn't have any parameters
- the function body is written inside `{}`

Function Declaration

Return type vs void function

11

- ▶ It can be void, which means function doesn't return any value.
- ▶ We have to omit return statement (Except the empty return)
- ▶ If function has a return type like int, string or float, then it should return a value of that type.
- ▶ Any code after return statement is not executed, and return can't be empty.

```
void displayNumber() {  
    // code  
}
```

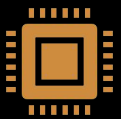
```
int add (int a, int b) {  
    return (a + b);  
}
```

Calling a function

12



We call the function by writing the function name.



If function returns some value, we store it in a variable or we can direct it to console screen.



If function doesn't return any value, we can simply write it's name only.

```
int main() {  
  
    // calling a function  
    greet();  
  
}
```

Example
of calling
a
function

How function works in C++

```
#include<iostream>

void greet() {
    // code
}

int main() {
    ... ..
    greet();
    ... ..
}
```

function
call

Example

15

```
// Online C++ compiler to run C++ program online
#include <iostream>
using namespace std;

int sum(int a, int b){
    return a+b;
}

int main() {
    int result = sum(4,5);
    cout << result;
    return 0;
}
```

Multiple Functions

16

```
// Online C++ compiler to run C++ program online
#include <iostream>
using namespace std;

int sum(int a, int b){
    return a+b;
}
int diff(int a, int b){
    return a-b;
}
int main() {
    int result = sum(4,5);
    int sub = diff(10,7);
    cout << result << endl;
    cout << sub;
    return 0;
}
```

Example (Void > no return)

17

```
// Online C++ compiler to run C++ program online
#include <iostream>
using namespace std;

void sum(int a, int b){
    cout << a+b;
}
int main() {
    sum(4,5);
    return 0;
}
```

Activity

DEFINE A FUNCTION NAMED AS FIND_MAX, WHICH FINDS OUT MAXIMUM NUMBER AMONG ANY TWO GIVEN NUMBERS AND RETURNS THE MAXIMUM NUMBER

```
#include <iostream>
using namespace std;

// Function to find maximum of two numbers
int find_max(int a, int b) {
    if (a > b)
        return a;
    else
        return b;
}

int main() {
    int num1, num2;

    cout << "Enter first number: ";
    cin >> num1;

    cout << "Enter second number: ";
    cin >> num2;

    int maximum = find_max(num1, num2);
    cout << "Maximum number is: " << maximum << endl;

    return 0;
}
```

Solution

Function prototype

20

- ▶ A function should be always defined before it is called
- ▶ However if you want to define a function later on you can use function prototype

```
returnType functionName(dataType1, dataType2, ...);
```

```
#include <iostream>
using namespace std;

// Function prototype (declaration)
int add(int a, int b);

int main() {
    int result = add(4, 5); // Function
    call
    cout << "Sum: " << result << endl;
    return 0;
}

// Function definition
int add(int a, int b) {
    return a + b;
}
```

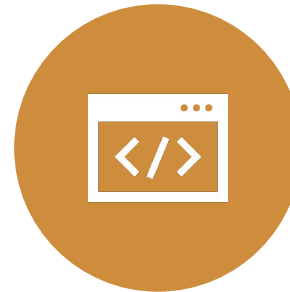
Function Prototype

Benefits

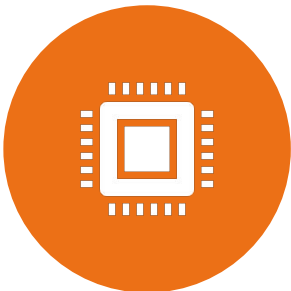
22



Allows calling functions before their definition — you can write `main()` first and define functions later.



Enables modular programming — keeps your code organized and easier to read.



Provides type checking — compiler verifies that function calls have correct argument types and return type.



Supports multi-file programs — functions can be declared in header files and defined elsewhere.

Parameters

23



Parameters are **variables declared in a function definition** to receive values from the caller.



They act as **placeholders** for the input data a function needs to work.



Defined inside **parentheses ()** after the **function name**.



Their **data types** must match the arguments passed during the function call.

Parameters



Formal parameters: Variables defined in the function header that receive values when the function is called.

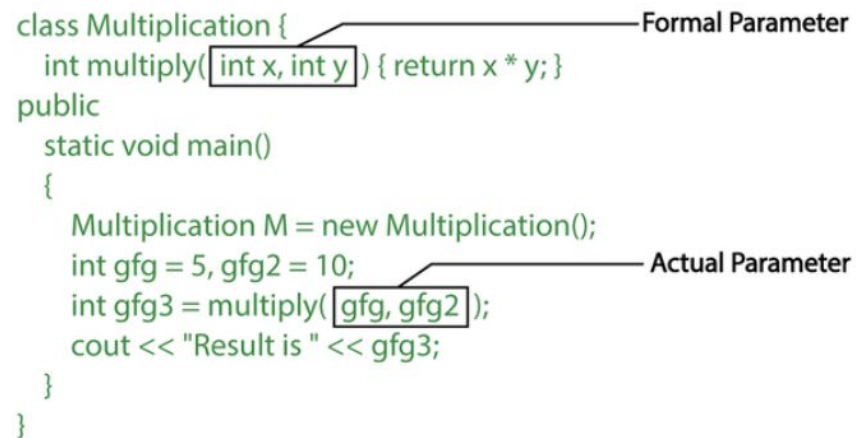


Actual parameters: Real values or variables passed to the function during the call.

The parameters passed to the function are called **actual parameters**. For example, in the program below, 5 and 10 are actual parameters.

The parameters received by the function are called **formal parameters**. For example, in the above program x and y are formal parameters.

```
class Multiplication {  
    int multiply(int x, int y) { return x * y; }  
public  
    static void main()  
    {  
        Multiplication M = new Multiplication();  
        int gfg = 5, gfg2 = 10;  
        int gfg3 = multiply(gfg, gfg2);  
        cout << "Result is " << gfg3;  
    }  
}
```



Formal Parameter and Actual Parameter

Formal Parameters and Actual Parameters

```
#include <iostream>
using namespace std;

// Function with default arguments
int add(int a, int b = 5) {
    return a + b;
}

int main() {
    cout << add(10) << endl;    // Uses default
    value of b = 5 → Output: 15

    return 0;
}
```

Function with default arguments

```
#include <iostream>
using namespace std;

// Function with default arguments
int add(int a, int b = 5) {
    return a + b;
}

int main() {
    cout << add(10) << endl;    // Uses default
    value of b = 5 → Output: 15
    cout << add(10, 20) << endl; // Overrides
    default → Output: 30
    return 0;
}
```

Overriding default parameter

```
#include <iostream>
using namespace std;

// Function to print elements of a fixed-size array
void printArray(int arr[5]) {
    for (int i = 0; i < 5; i++)
        cout << arr[i] << " ";
}

int main() {
    int numbers[5] = {10, 20, 30, 40, 50};

    printArray(numbers); // Call function

    return 0;
}
```

C++ Passing an array to function

```
#include <iostream>
using namespace std;

// Function to print elements of a fixed-size array
void printArray(int arr[5]) {
    int sum = 0;
    for (int i = 0; i < 5; i++){
        cout << arr[i] << " ";
        sum +=arr[i];
    }
    cout << "sum : " << sum << endl;
}

int main() {
    int numbers[5] = {10, 20, 30, 40, 50};
    printArray(numbers); // Call function

    return 0;
}
```

Example

C++ Passing an array to function

30

- ▶ Write a C++ code to create a function which can find count of even numbers from an array of 10 elements

```
#include <iostream>
using namespace std;

// Function to print elements of a fixed-size array
int printArray(int arr[10]) {
    int c = 0;
    for (int i = 0; i < 10; i++){
        if (arr[i]%2==0){
            c=c+1;
        }

        cout << arr[i] << " ";
    }
    return c;
}

int main() {
    int numbers[10] = {1,2,4,4,8,6,7,8,9,10};

    int evenCount = printArray(numbers); // Call function
    cout << evenCount;

    return 0;
}
```

Solution

Recap

32

- ▶ Function
- ▶ Function Declaration/Prototype
- ▶ Function Definition
- ▶ Calling a Function
- ▶ Return type vs void
- ▶ Parameters
- ▶ cin vs parameters
- ▶ Formal parameters vs Actual Parameters
- ▶ Function with default arguments
- ▶ Passing an array to the function

Function Overloading

- ▶ Function overloading means when two or more functions have same name but different parameters.
- ▶ Parameters can be different in terms of:
 - ▶ Number
 - ▶ Data Type
 - ▶ Sequence

```
#include <iostream>
using namespace std;

int add(int a, int b) {
    return a + b;
}

int add(int a, int b, int c) {
    return a + b + c;
}

int main() {
    cout << add(5, 10) << endl;    // Calls first function → 15
    cout << add(5, 10, 15) << endl; // Calls second function
    → 30
    return 0;
}
```

Different
number of
parameters

```
#include <iostream>
using namespace std;

int add(int a, int b) {
    return a + b;
}

double add(double a, double b) {
    return a + b;
}

int main() {
    cout << add(5, 10) << endl;    // int version → 15
    cout << add(5.75, 10.5) << endl; // double version →
16.0
    return 0;
}
```

Different Data Type of parameters

```
#include <iostream>
using namespace std;

void display(int a, double b) {
    cout << "int first: " << a << ", double second: " << b <<
endl;
}

void display(double a, int b) {
    cout << "double first: " << a << ", int second: " << b <<
endl;
}

int main() {
    display(5, 3.5); // Calls first version
    display(3.5, 5); // Calls second version
    return 0;
}
```

Different Sequence of parameters

Class Activity

37

- ▶ Create a function named `names_arr`, this function should return an array of 5 strings containing names of persons.

```
helloWorld.cpp  scope.cpp
1  #include<iostream>
2  using namespace std;
3
4  double calculateRadius(double r) {
5      return r * r;
6  }
7
8  double calculateArea(double r) {
9      double radiusSquared = calculateRadius(r);
10     return radiusSquared * 3.14159;
11 }
12
13 int main() {
14     double radius = 5.0;
15     double area = calculateArea(radius);
16     cout << "The area of the circle is: " << area << std::endl;
17     return 0;
18 }
```

C:\Users\LENOVO\Documents x + -
The area of the circle is: 78.5397

Function
inside
function

Math Library

39

- ▶ Include Library using `#include<cmath>`
- ▶ Few Functions:
 - ▶ `double sin(double)`
 - ▶ `double cos(double)`
 - ▶ `double tan(double)`
 - ▶ `double sqrt(double)`
 - ▶ `double pow(double,double)`
 - ▶ `double floor(double)`
 - ▶ `double ceil(double)`
 - ▶ `double round(double)`
 - ▶ `Int abs(int)`

Explore math library

40

- ▶ [https://www.programiz.com/cpp-programming/library-function/c-math/round#:~:text=The%20round\(\)%20function%20in,in%20the%20cmath%20header%20file.](https://www.programiz.com/cpp-programming/library-function/c-math/round#:~:text=The%20round()%20function%20in,in%20the%20cmath%20header%20file.)

Function inside function

- ▶ Functions can call other functions and a function can call itself. This allows you to organize your code and reuse functionality.

```
#include <iostream>
#include<array>
using namespace std;

std::array<int,5> func() //function with return type std:
{
    std::array<int,5> f_array; //array declared

    for(int i=0;i<5;i++)
    {
        //array initialisation
        f_array[i] = i;
    }

    return f_array; //array returned
}
```

Return
an array
from the
function

```
int main()
{
    std::array<int,5> arr; //array with length

    arr=func(); //function call

    cout<<"The Array is : ";
    for(int i=0;i<5;i++)
    {
        cout<<arr[i]<<"\t";
    }

    return 0;
}
```

Return
an array
from the
function

Class Activity

44

- ▶ Define a function which returns square root of a given number

Class Activity

45

- ▶ Define a function which takes an array of temperature values in Kelvin as its parameter and then prints each of the value in Celsius on console screen.
- ▶ $C = K - 273.15$

Type conversion will also work with
functions

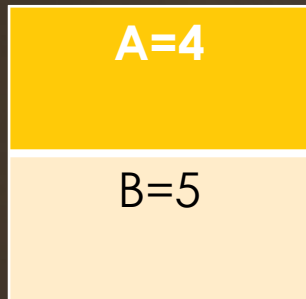
Call by value vs Call by reference

- ▶ Function can be invoked/called in two ways:
 - ▶ Call by value
 - ▶ Call by reference

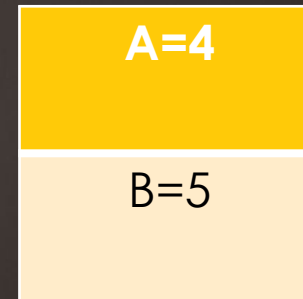
Call by value

48

- ▶ In call by value:
 - ▶ Actual parameters are copied to functions formal parameters
 - ▶ There are two copies of parameters at different memory locations:



Original
Copy

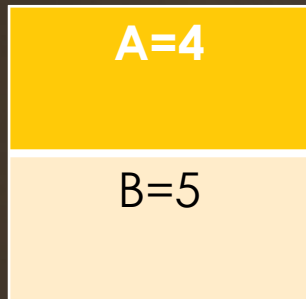


Function
Copy

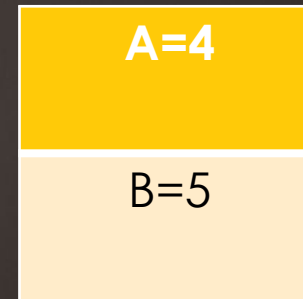
Call by value

49

- ▶ In call by value:
 - ▶ Any changes made inside the function are made into the function copy
 - ▶ While original copy remains unaffected in the calling function



Original
Copy



Function
Copy

Example: Swap two values

```
#include<iostream>
using namespace std;

void swap(int a,int b){
    int t;
    t=a;
    a=b;
    b=t;
    cout<<"Inside called function: "<<"a="<<a<<", b="<<b<<endl;
}

int main(){
    int a=4;
    int b=5;
    swap(a,b);
    cout<<"Inside calling function: "<<"a="<<a<<", b="<<b;

    return 0;
}
```

Call by Reference

- ▶ In call by reference:
 - ▶ Address of the actual parameters is passed to the formal parameters
 - ▶ Now formal and actual parameters have a single copy
 - ▶ Any changes made inside the function are also visible in the calling function

Example: Swap two values

```
#include<iostream>
using namespace std;

void swap(int* a,int* b){
    int t;
    t=*a;
    *a=*b;
    *b=t;
}
```

```
int main(){
int a=4;
int b=5;
swap(a,b);
cout<<a<<endl<<b;
return 0;
}
```

Passing an array by reference

- ▶ Remember to fix the size of array

```
#include<iostream>
using namespace std;

void array_by_reference(int (&arr)[8]){

    for(int &element:arr){
        element+=1;
    }
}
```

Passing an array by reference

- ▶ Remember to fix the size of array

```
int main(){  
    int my_arr[]={1,2,3,4,5,6,7,8};  
    array_by_reference(my_arr);  
    for(int element:my_arr){  
        cout<<element<<"\n";  
    }  
  
    return 0;  
}
```

- ▶ Define a function which takes value in kilometers as its parameter and converts it into meters and display it on console screen.
- ▶ Define a function which takes an array as its parameter and returns an array which contains two values, first value of the returned array is the sum of even numbers from the parameter array, and second value is the sum of the odd numbers from the parameter array.
- ▶ Define a function which calculates area of a square, pass appropriate parameters to this function
- ▶ Define a function which takes height of a person in feet and returns the height value in centimeter
- ▶ Define a function which takes two arrays as parameters, first names array and last names array and display the first name and last name concatenated on the screen,

Recursion

56

- ▶ Technique to make a function call itself.
- ▶ It is a way to break complicated problems into simple problems which are easier to solve

A Mathematical Interpretation

57

- ▶ Let us consider a programmer has to determine the sum of first n natural numbers:

approach(1) – Simply adding one by one

$$f(n) = 1 + 2 + 3 + \dots + n$$

approach(2) – Recursive adding

$$f(n) = 1 \quad n=1$$

$$f(n) = n + f(n-1) \quad n>1$$

Solving the recursion

58

- ▶ $f(5)=5+f(5-1)$ -----eq:01
- ▶ $f(5-1)=f(4)=4+f(4-1)$ -----eq:02
- ▶ *Substitute 02 in 01*
- ▶ $f(5)=5+4+f(4-1)$ -----eq:03
- ▶ $f(4-1)=f(3)=3+f(3-1)$ -----eq:04
- ▶ *Substitute 04 in 03*
- ▶ $f(5)=5+4+3+f(3-1)$ -----eq:05
- ▶ $f(3-1)=f(2)=2+f(2-1)$ -----eq:06
- ▶

Solving the recursion

59

- ▶ *Substitute 06 in 05*
- ▶ $f(5)=5+4+3+2+f(2-1)$ -----eq:07
- ▶ $f(2-1)=f(1)=1$ -----eq:08
- ▶ *Substitute 08 in 07*
- ▶ $f(5)=5+4+3+2+1$
- ▶ $f(5)=15$

Solution to add the numbers

60

```
int fact(int n)
{
    if (n <= 1) // base case
        return 1;
    else
        return n*fact(n-1);
}
```

Factorial

61

- ▶ Try Factorial with Recursion

```
int factorial(int n) {  
    if (n == 0) {  
        return 1;  
    } else {  
        return n * factorial(n - 1);  
    }  
}
```

Solution